Iterative Improvement: Maximum Matching in Bipartite Graphs

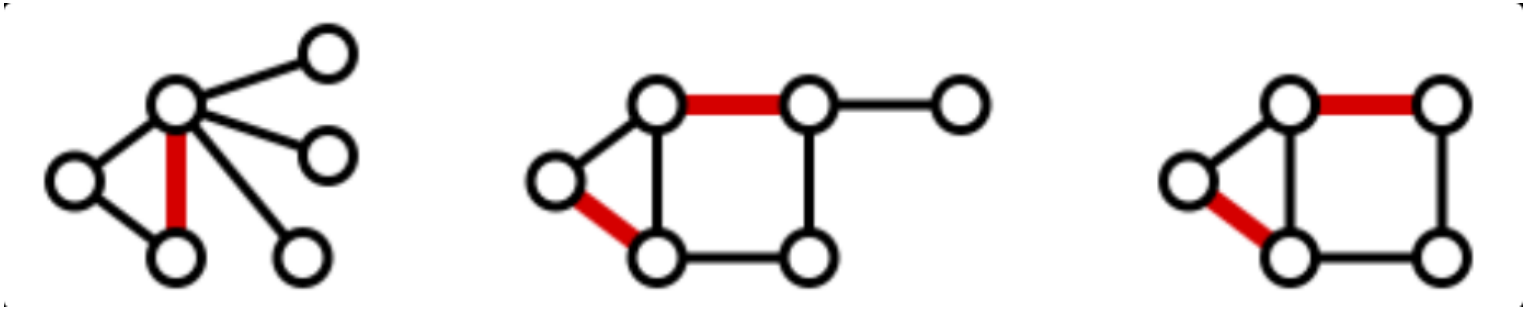
Design and Analysis of Algorithms

Dhannya SM

Dept of CSE
SSN College of Engineering

Matchings in a graph [1]

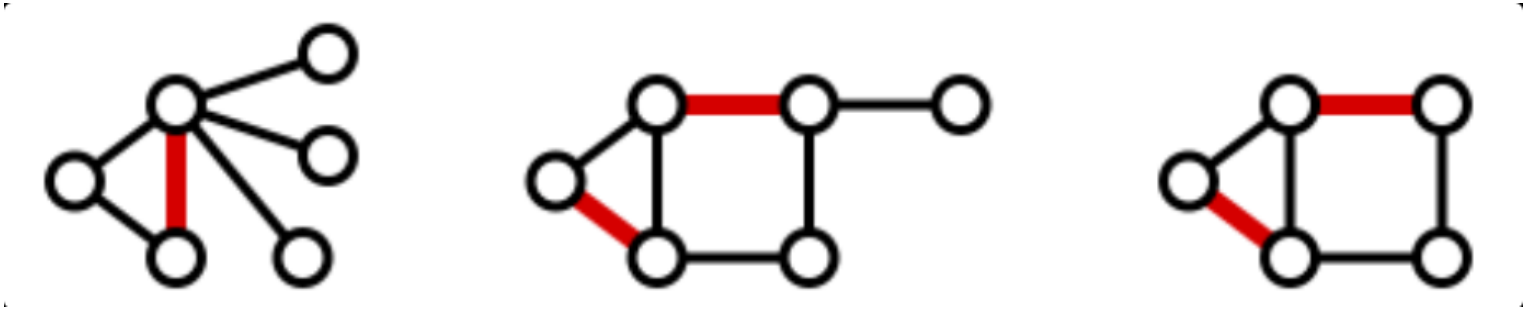
- A *matching* in a graph is a subset of its edges with the property that no two edges share a vertex.



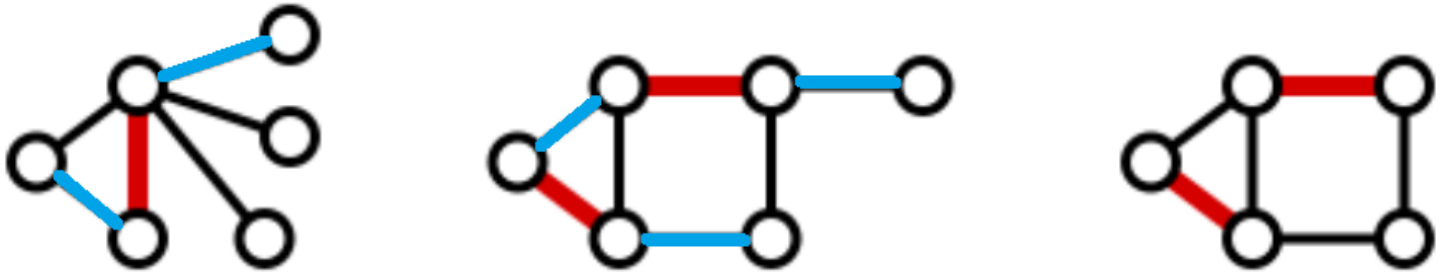
[1] www.wikiwand.com

Matchings in a graph [1]

- A *matching* in a graph is a subset of its edges with the property that no two edges share a vertex.



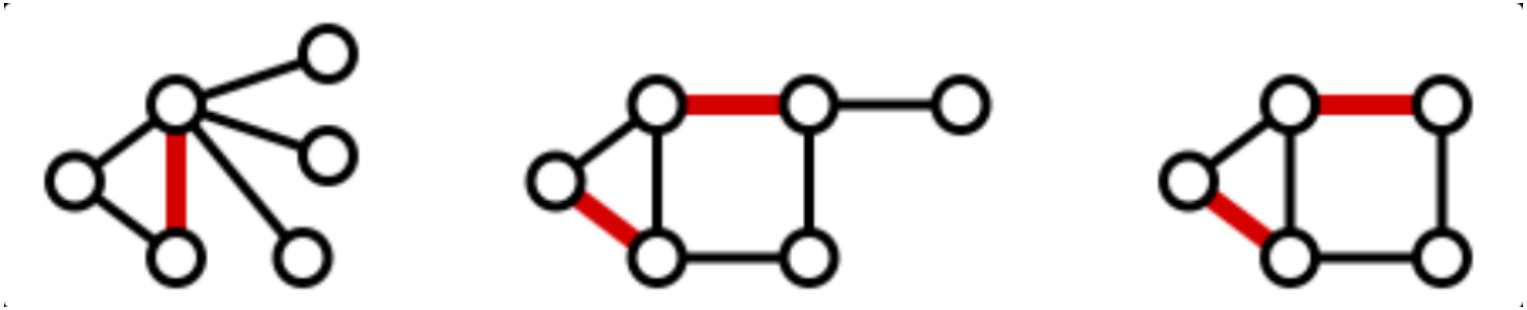
- A *maximum matching* (or a maximum cardinality matching) is a matching with the largest number of edges.



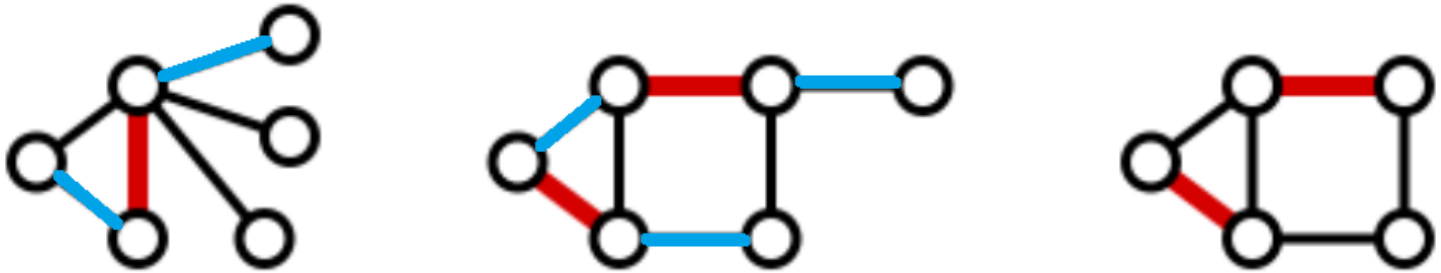
[1] www.wikiwand.com

Matchings in a graph [1]

- A *matching* in a graph is a subset of its edges with the property that no two edges share a vertex.



- A *maximum matching* (or a maximum cardinality matching) is a matching with the largest number of edges.

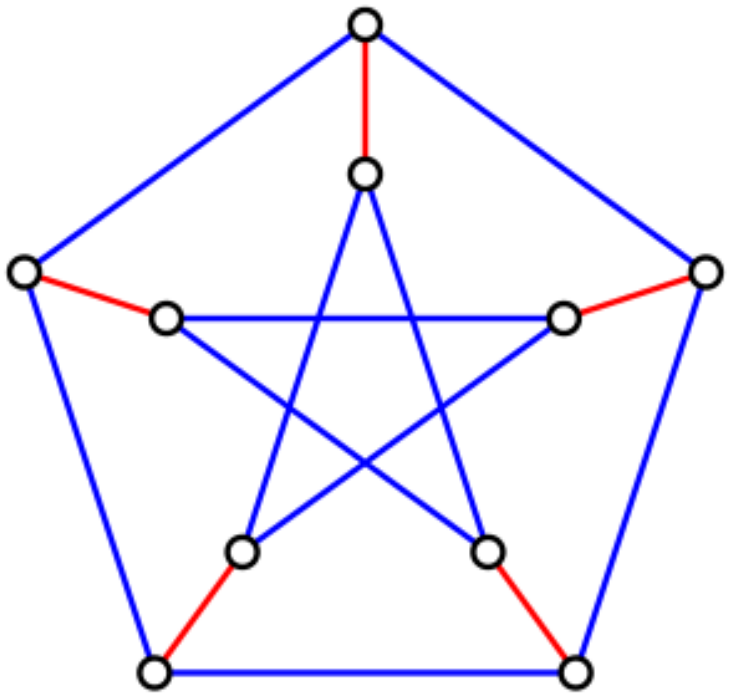


- The *maximum-matching problem* is the problem of finding a maximum matching in a given graph.

[1] www.wikiwand.com

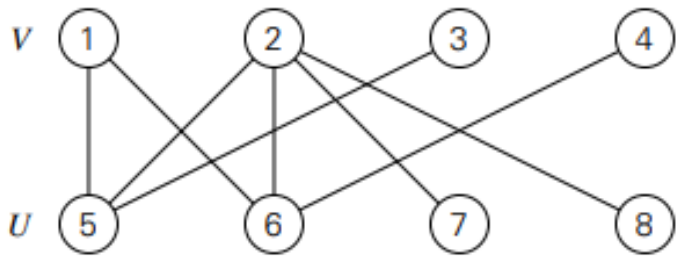
Perfect matching

- A *perfect matching* of a graph is a matching in which every vertex of the graph is incident on exactly one edge of the matching. More formally, given a graph $G = (V, E)$, a perfect matching in G is a subset M of edge set E, such that every vertex in the vertex set V is adjacent to exactly one edge in M.

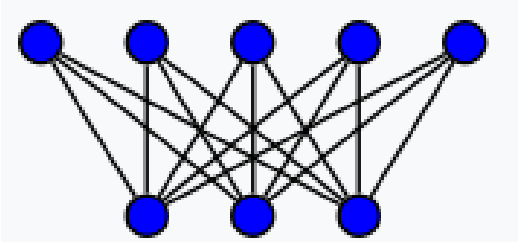


Bipartite graphs

- In a *bipartite graph*, all the vertices can be partitioned into two disjoint sets V and U, not necessarily of the same size, so that every edge connects a vertex in one of these sets to a vertex in the other set.



- A graph is bipartite if its vertices can be colored in two colors so that every edge has its vertices colored in different colors; such graphs are also said to be *2-colorable*.
- A complete bipartite graph is a bipartite graph in which every vertex in set V is connected to every vertex in U.

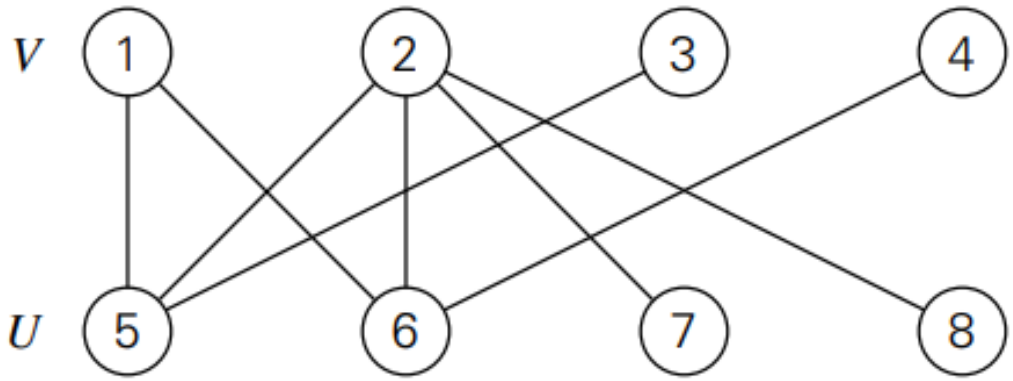


Matchings in bipartite graphs

- The problem of finding a maximum matching in a bipartite graph has many practical applications.
- Consider matching a set L of machines with a set R of tasks to be performed simultaneously. An edge (u, v) signifies that a particular machine $u \in L$ is capable of performing a particular task $v \in R$. A maximum matching provides work for as many machines as possible.
- There are $|L|$ job applicants and $|R|$ jobs. Each applicant has a subset of jobs that he/she is qualified to do. Each job vacancy can only accept one applicant and each job applicant can be appointed for only one job. A maximum matching will help fill maximum job vacancies.
- Other applications include matching scholars to advisors, users to servers etc.

Maximum Bipartite Matching

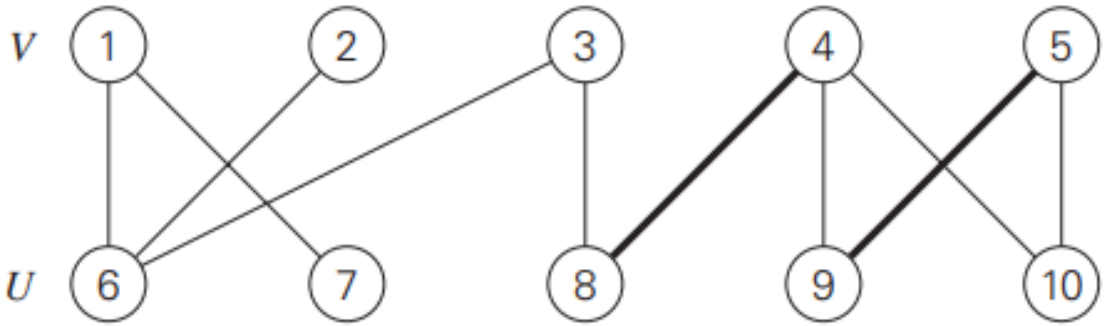
- Let $G = (V, E)$ be a bipartite graph where $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$ and there does not exist an edge $(u, v) \in E$ such that both u and v belong to either V_1 or V_2. That is, all edges in G go between V_1 and V_2.



- The problem of Maximum Bipartite Matching seeks to find a maximum matching in G.

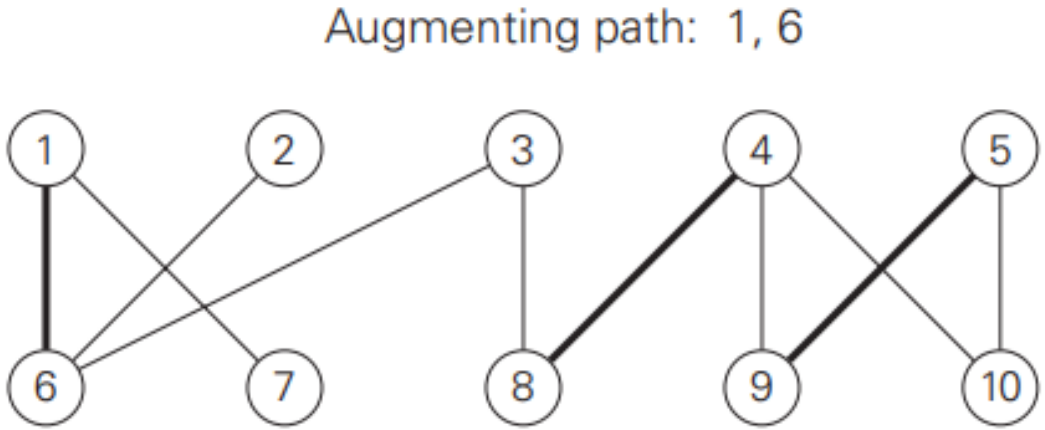
Iterative improvement

- Let M be a matching in a bipartite graph $G = \langle V, U, E \rangle$.
- If every vertex in either V or U is matched, i.e., serves as an endpoint of an edge in M, then M is a maximum matching.
- Therefore, to have a chance at improving the current matching, both V and U must contain *unmatched* (also called free) vertices, i.e., vertices that are not incident to any edge in M.
- For example, in the graph given below, for the matching $M_1 = \{(4, 8), (5, 9)\}$, vertices 1, 2, 3, 6, 7, and 10 are free, and vertices 4, 5, 8, and 9 are matched.



Iterative improvement

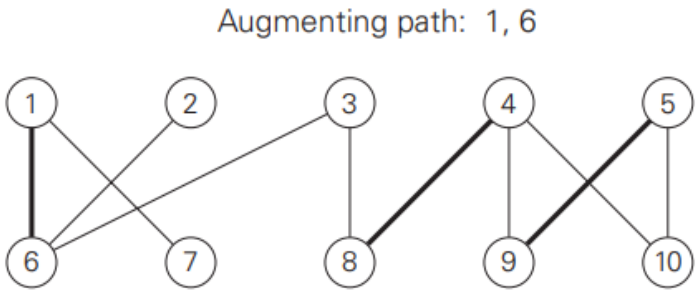
- We can immediately increase a current matching by adding an edge between two free vertices.



- The new matching is $M_2 = \{(1,6), (4, 8), (5, 9)\}$.

Iterative improvement

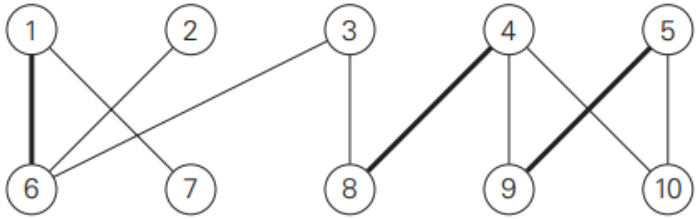
- Is there a larger matching?



Iterative improvement

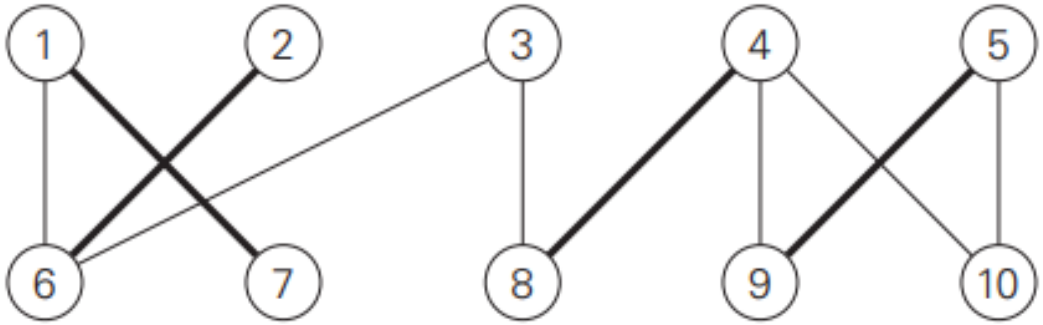
- Is there a larger matching?

Augmenting path: 1, 6



- Can we include the edge (2, 6) in a new matching?
- Yes. However, this inclusion requires removal of (1, 6), which can be compensated by inclusion of (1, 7) in the new matching.
- This new matching is $M_3 = (1, 7), (2, 6), (4, 8), (5, 9)$.

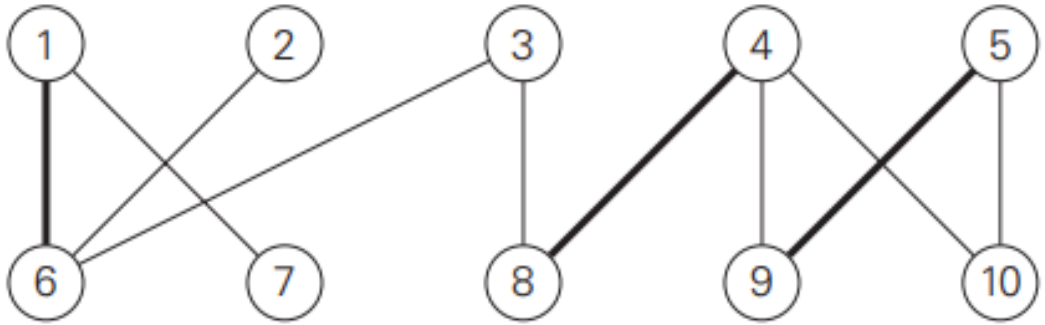
Augmenting path: 2, 6, 1, 7



Iterative improvement

- We increase the size of a current matching M by constructing a simple path from a free vertex in V to a free vertex in U whose edges are alternately in $E - M$ and in M.
- That is, the first edge of the path does not belong to M, the second one does, and so on, until the last edge that does not belong to M.
- Such a path is called *augmenting* with respect to the matching M.
- The path 2, 6, 1, 7 is an augmenting path with respect to the matching in the figure below.

Augmenting path: 1, 6

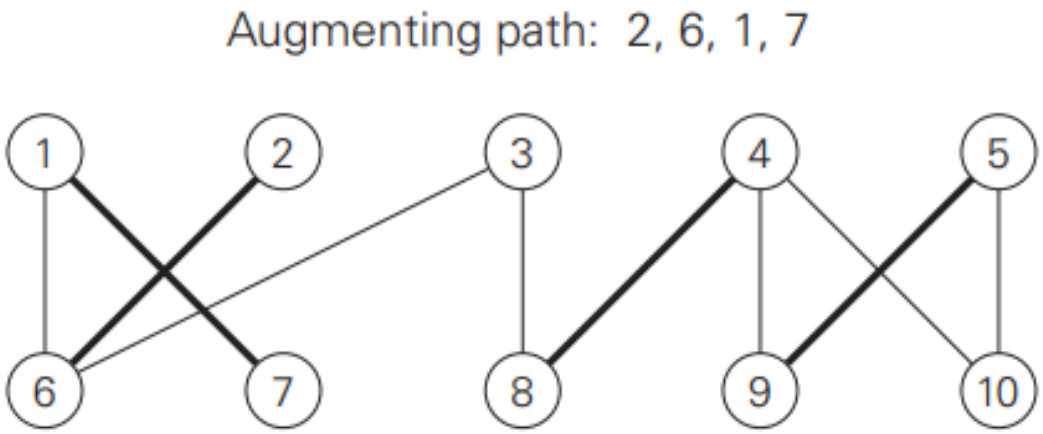


Iterative improvement

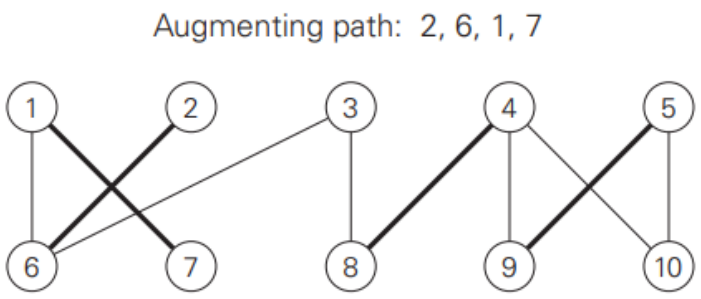
- Observe that the length of an augmenting path is always odd. (Why?)
- Since the length of an augmenting path is always odd, adding to the matching M the path's edges in the odd-numbered positions and deleting from it the path's edges in the even-numbered positions yields a matching with one more edge than in M.
- Such a matching adjustment is called augmentation.

Iterative improvement

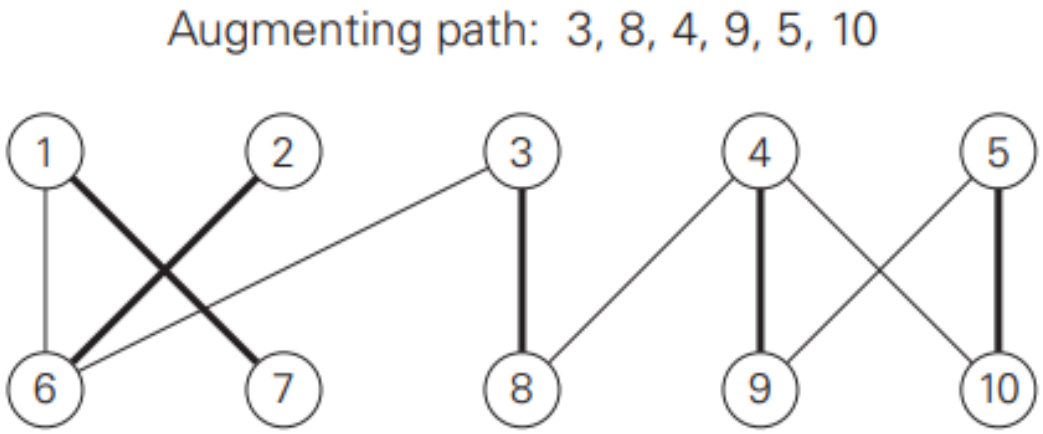
- Proceeding further, 3, 8, 4, 9, 5, 10 is an augmenting path for the matching M_3.



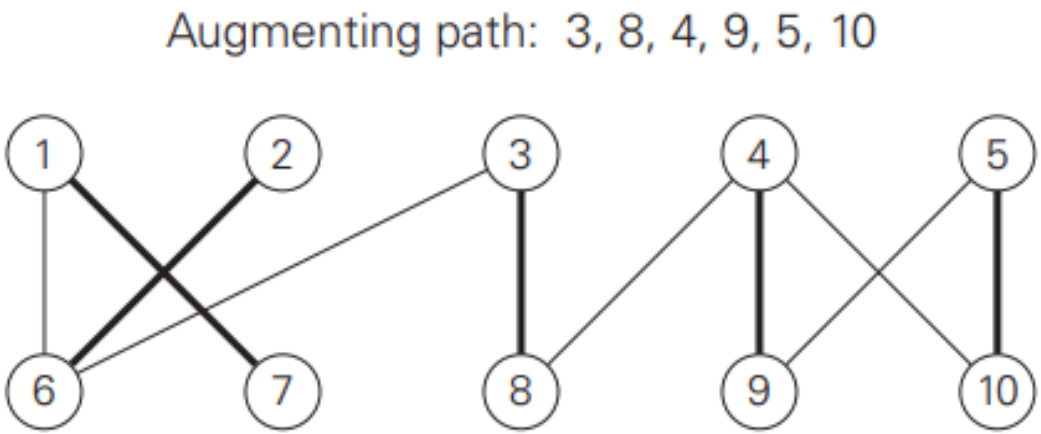
Iterative improvement



- After adding to M_3 the edges (3, 8), (4, 9), and (5, 10) and deleting (4, 8) and (5, 9), we obtain the matching $M_4 = (1, 7), (2, 6), (3, 8), (4, 9), (5, 10)$.



Iterative improvement



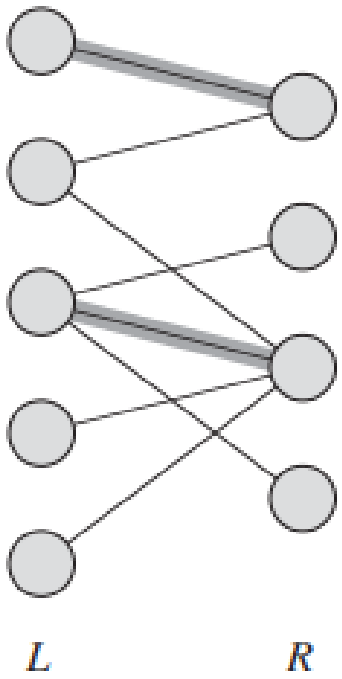
- Observe that M_4 is not only a maximum matching but also a *perfect* one, i.e., a matching that matches all the vertices of the graph.

Theorem: A matching M is a maximum matching if and only if there exists no augmenting path with respect to M.

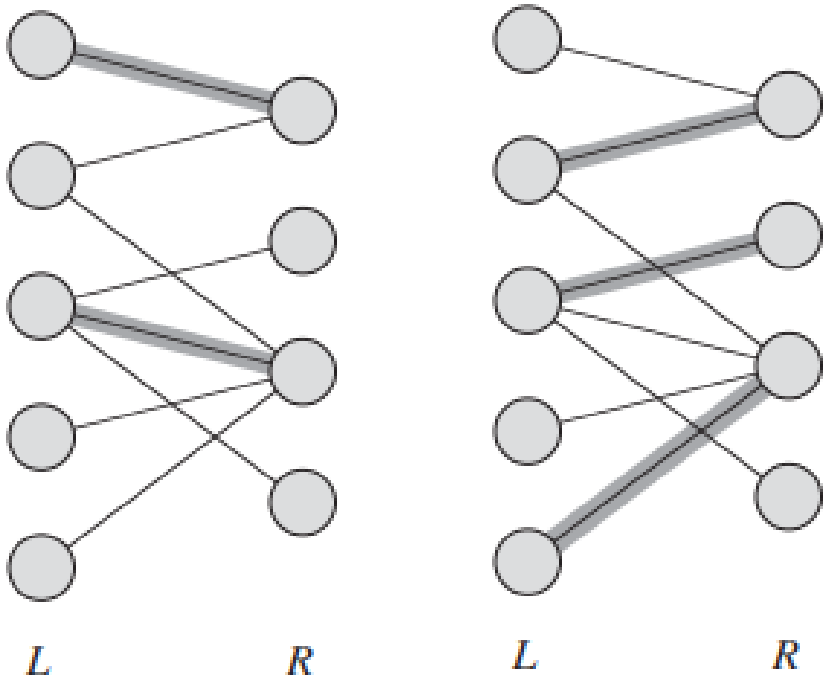
Ford-Fulkerson Method

- We can use the Ford-Fulkerson method to find a maximum matching in an undirected bipartite graph.
- The idea is to construct a flow network in which flows correspond to matchings.

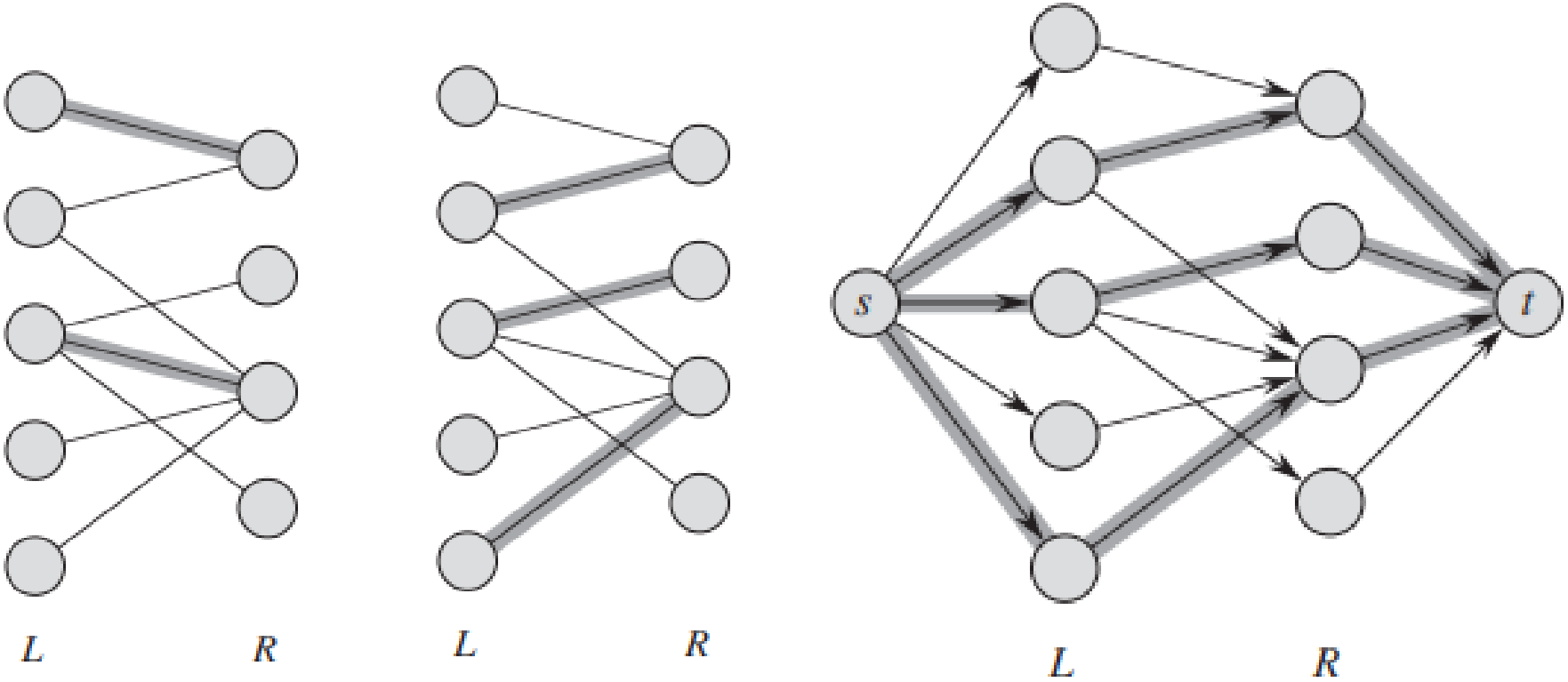
Ford-Fulkerson Method



Ford-Fulkerson Method



Ford-Fulkerson Method



Summary

- The problem of Maximum Bipartite Matching seeks to find a maximum matching in G.
- General method:
  - Start with some initial matching (possibly an empty set).
  - Find an augmenting path and augment the current matching along this path.
  - When no augmenting path can be found, terminate the algorithm and return the last matching, which is maximum.